

CARD-ON-FILE *tokenization* MIGRATION PLAN

Moving member billing off stored card and bank numbers and onto IntelliPay tokens — the one-time sweep, the data-model change, the nightly batch, and what it means for online joins.

Prepared **2 Sep 2026**

Processor **IntelliPay**

For **GHF IT · member-system vendor · IntelliPay**

WHERE WE ARE, *where we're going*

The member system currently holds full card and bank numbers and ships them to IntelliPay in the nightly file. After this project it holds a token and nothing else.

TODAY

- Full card numbers, expiry, bank account & routing numbers stored in the member system
- Nightly SFTP batch rows carry the raw numbers
- Front desk keys numbers into a text field
- GHF holds cardholder data — full PCI scope (SAQ-D territory)

AFTER

- One
TOKEN per member plus display-only hints (brand, last 4)
- Batch rows carry the token
- Staff and members add cards through IntelliPay's capture; the system only ever receives a token
- GHF never touches a number — SAQ-A territory

custid → 19682099

A token is IntelliPay's **customer id**. It's scoped to GHF's merchant account, useless to anyone else, works for cards and bank accounts alike, and every IntelliPay charge path — `card_payment`, `payment_create`, `batch_upload` — accepts it in place of a number.

WHAT TO *store*

Add six fields, remove the numbers, keep the display data. The "Credit Card #" box becomes a read-only TOKEN field with an "Add / replace payment method" button beside it.

CHANGE	FIELD	TYPE	NOTES
ADD	TOKEN	string(32)	IntelliPay <code>custid</code> . Required for anyone on autopay. Read-only in the UI — only the system writes it.
ADD	TOKEN_TYPE	C / A	Card or bank (ACH). The batch file needs it per row.
ADD	METHOD_HINT	string(24)	<code>VI ***1111</code> or <code>Bank ***4321</code> . Display only.
ADD	TOKEN_SOURCE	enum	<code>migration</code> · <code>online-join</code> · <code>front-desk</code> · <code>member-update</code>
ADD	TOKEN_CREATED	datetime	When IntelliPay issued it.
ADD	TOKEN_STATUS	enum	<code>active</code> · <code>expired</code> · <code>declined</code> · <code>needs-update</code> — driven by batch return codes.
REMOVE	CARD_NUMBER, EXP, CVV	—	After the sweep is verified (Phase 5). CVV should never have existed.
REMOVE	BANK_ACCT, ROUTING	—	Same. And everywhere else they live: reports, spreadsheets, exports, old batch files on the SFTP server, logs, database backups .
KEEP	NAME, ADDRESS, BRAND, LAST4, BANK_NAME	—	Not cardholder data. Needed for display and for reconciling the sweep.

THE ONE-TIME *sweep*

Every active billing record needs a token before the first token-based dues run. Two ways to get there — ask for the first one before building the second.

OPTION A

RECOMMENDED

IntelliPay bulk import

Processors run a card-data import for exactly this: GHF hands over one **encrypted file** (PGP over SFTP), IntelliPay tokenizes on their side and returns a mapping of *member ID* → *custid*.

- GHF writes no code that handles numbers; the export is one controlled event.
- Ask for: file spec, encryption key, ACH handling, turnaround, and a return file keyed on our member ID via the `account` field.

API sweep with `cust_create`

The API accepts raw card or bank fields and returns the `custid`. GHF already holds the numbers, but IntelliPay's own guidance permits this **only for a PCI-validated merchant** — otherwise it's a violation even though the API accepts it.

- One POST per member: `account` =member ID, name, address (state 2-letter, country `USA`), then card (`cardnum` , `expdate` MMY) or bank (`bankacctnum` , `routingnum` , type C/S).
- Locked-down host, TLS only, request bodies never logged, run once.

Either way — the sweep checklist

1 Freeze payment-method edits
in the member system for the sweep window — between two dues Wednesdays.

2 Clean the data first.
This is where migrations fail.

- De-duplicate: one billing record per member; `account` must be unique — it's how we reconcile.
- Normalize `expdate` to MMY, state to 2 letters, country to ISO-3.
- Luhn-check card numbers; ABA-checksum routing numbers; bank type C/S.
- Pull out already-expired cards.
IntelliPay will store them, but every charge fails (`-7 stored card expired`). Those members go straight to outreach.
- Members with both a card and a bank account: pick the primary; create one token each if both are needed.

3 Rehearse in sandbox
with IntelliPay test cards and a 50-row sample. Confirm the return mapping lands in `TOKEN` correctly.

4 Run production.
Capture a per-row result: `custid` or the failure reason.

5 Reconcile.
Every active autopay member has a token or sits on a named exception list with a reason. Retry transient failures only after `cust_search(account=...)` so nothing is created twice.

6 Outreach
to the exception list — expired, invalid, missing. Text or email a secure "update your payment method" link (Lightbox Store Only) or handle at the desk, with a deadline before the first token-based dues run.

THE NIGHTLY *SFTP batch*

Today the file carries numbers; after cutover it carries token, type, amount, and invoice.
IntelliPay's batch upload is the same processing path as `payment_create` , so its rules and return codes apply.

ITEM	WHAT TO DO
Get the token-based file spec	Column names for custid / type / amount / invoice, delimiter, and the rejection/return file format. BLOCKING
Unique invoice per member per cycle	e.g. <code>GHF-<memberId>-<YYYYMMDD></code> . Lets us search before resubmitting and never double-charge.
Map return codes to <code>TOKEN_STATUS</code>	<code>-2</code> custid not found (missing token or wrong merchant) · <code>-7</code> card expired · <code>-8</code> stored card missing · <code>-4</code> / <code>-5</code> bank details invalid or missing · <code>-10</code> amount problem. Each triggers an outreach action.
ACH timing	ACH rows are voidable until 5 pm MT on the day the file is transmitted; card rows settle per normal.
Parallel cycle	For one dues cycle, generate the file both ways, compare row counts and totals, then transmit only the token file.
Retire the old files	Delete historical batch files containing numbers from the SFTP server and any archive.

Multiple clubs. A `custid` belongs to one IntelliPay merchant account. If Main, Women, and Tioga bill under separate merchant IDs, each member needs a token under the MID that bills them. Confirm the MID structure before the sweep — a `-2` in the returns is the tell when it's wrong.

WHAT HAPPENS WITH *online joins*

Nothing changes in principle. The join flow we built is already token-native — IntelliPay's Lightbox captures the card in the browser and hands us the token.

DETAILS

Member picks club and plan, enters details. We create the member record and a **member ID**.

RECURRING METHOD

Lightbox in Store Only mode. Card or bank data goes browser → IntelliPay. We get `custid` , type, hint → `TOKEN` , `TOKEN_TYPE` , `METHOD_HINT` , source `online-join` .

DUE TODAY

One real charge by `custid` — or a separate card if they chose bank draft. GHF never sees a number.

THAT NIGHT

The member is in the export with their token — same file, same columns as migrated members.

Two things to lock down

We pass the **member ID as IntelliPay's `account`** field, so `cust_search(account=...)` always finds the right token — the same convention the sweep uses.

The **front desk must stop keying numbers** on cutover day. Give staff the same tool the website uses: a staff-facing Lightbox Store Only page, or the Lightbox Card Present terminal for chip and tap, either of which hands the member system a token. One card typed into the old field puts GHF back in scope.

KEEPING TOKENS *healthy*

Visa Account Updater	Turn it on. Monthly, IntelliPay checks stored Visa/Mastercard cards expiring next month against the card networks and refreshes reissued numbers and expiry automatically. Needs a saved method on an active, primary customer record. Amex and Discover aren't covered — keep the expiring-card outreach for those.
Postbacks	IntelliPay's server-to-server confirmation is the authoritative record; feed it into <code>TOKEN_STATUS</code> .
Member self-service	An "update my card" link that opens Lightbox Store Only and replaces the token — no staff, no numbers.
Access control	Tokens are merchant-scoped and not cardholder data, but treat them as sensitive: role-based access, audit log on changes.

PHASED *timeline*

Six phases. The sweep sits between two dues Wednesdays so no member is billed mid-migration.

PHASE · WHEN	
WHAT	DONE WHEN
0 Discovery Week 1 Confirm GHF's PCI status (is there an AOC?). Count active autopay members — card vs bank, per club/MID. Open the IntelliPay questions. Order front-desk terminals. DONE WHEN Answers in hand; Option A or B chosen; terminals ordered.	

1

Build

Weeks 1–3

Vendor adds TOKEN fields, read-only UI, the "Add / replace method" launcher; batch generator emits tokens; online-join export maps to the new fields; staff join page in `businessattended` mode.

DONE WHEN

Sandbox end-to-end: join → token → batch file → IntelliPay accepts.

2

Rehearse

Week 3

50-row sample sweep in sandbox; token batch file accepted; return handling tested with forced `-2` / `-7`.

DONE WHEN

Runbook written; timings known.

3

Sweep

Between two dues Wednesdays

Freeze edits → clean data → run → reconcile → outreach list.

DONE WHEN

100% of active autopay have a token or a named exception.

4

Cutover

Next dues cycle

Parallel file compare → transmit the token file → monitor returns for 48 h. Front desk switches to terminal capture the same day.

DONE WHEN

First token cycle settles; return rate $\approx$ historical.

5

Purge & attest

Within 30 days

Delete numbers from DB, reports, exports, SFTP archive, logs, and backups (or set retention so they expire). Document destruction. Rotate IntelliPay API keys. Update the PCI self-assessment.

DONE WHEN

Written attestation that no cardholder data remains.

RISKS AND *mitigations*

RISK	MITIGATION
Expired or invalid cards surface in the sweep	Expected — that is the exception list. Outreach starts the day of the sweep with a deadline before Phase 4.
Duplicate member records → duplicate tokens	De-dupe first; <code>account</code> unique; <code>cust_search</code> before any retry.
Numbers survive in a backup or old report	Phase 5 inventory covers backups, SFTP archive, BI extracts, staff spreadsheets. Set backup retention so old copies age out.
Staff keep typing card numbers	Remove the field entirely; give them the Lightbox tool; brief and audit in the first week after cutover.
Token created under the wrong merchant ID	Confirm MID structure in Phase 0; sweep per MID.
Batch file rejected on the first token cycle	Parallel cycle in Phase 4; IntelliPay reviews a sample file before go-live.
ACH can't be tested in sandbox today	Ask IntelliPay to enable ACH on the test merchant (currently off).

OPEN *questions*

FOR INTELLIPAY

1. Do you offer a **bulk card/ACH import** (encrypted file → tokens)? Spec, encryption, turnaround, return mapping by `account` .
2. **Batch upload spec** for token rows (custid + type + amount + invoice) and the return file format.
3. Merchant-ID structure for the three clubs — can one `custid` bill under all?
4. Enable **Visa Account Updater**, and ACH on the sandbox merchant.
5. Production keys; does the join domain need registering in the iframe allow-list?
6. Production postback endpoint registration.
7. **Terminals:** recommended Dejavoo model, price per unit, monthly fees, lead time, and one merchant-key/API-key pair per desk (5).

FOR GHF AND THE MEMBER-SYSTEM VENDOR

1. Current PCI status — is there an AOC covering stored card data? Decides Option A vs B and how urgent Phase 5 is.
2. Every place a card or bank number is stored or printed today.
3. Active autopay count — card vs bank, per club.
4. Who owns the batch generator and the SFTP job; how fast can the mapping change?
5. Dues calendar — which two Wednesdays bracket the sweep window.

TERMINALS FOR THE *front desk*

After cutover nobody at a desk can type a card number into the member system, so every staffed desk that joins members or updates payment methods needs an IntelliPay-connected way to capture a card and hand back a token.

How it works

IntelliPay's **Lightbox Card Present Terminal** is the same Lightbox we use online, opened with `operatingenv=businessattended` . The form gains chip / tap / key-entry tabs and routes the capture through the physical terminal bound to that desk. Each terminal is bound to its own **merchant-key + API-key pair** in the IntelliPay admin, so the staff join page picks the desk's key pair and the token comes back exactly as it does online.

Hardware IntelliPay supports

Dejavoo tap-to-pay countertop	Chip + contactless, network-connected. PREFERRED — no card data touches the PC.
-------------------------------	--

Ingenico terminals	Same integration path.
ID Tech Augusta / 3350	USB chip readers with an IntelliPay driver, for a reader on the desk PC.

Use chip and contactless. Avoid magnetic swipe — unless the reader is P2PE-certified, swipe data puts PCI scope back on GHF.

What to buy

LOCATION	DESKS	UNITS
GHF Main (24/7)	Front desk + membership office	2
GHF Women	Front desk	1
GHF Tioga	Front desk	1
Spare / swap	—	1
Total		5

- Ask IntelliPay for a quote on the Dejavoo model they currently recommend, plus any per-terminal monthly gateway fee. Countertop EMV/contactless units typically run a few hundred dollars each — a budgeting placeholder until the quote lands.
- **Order in Phase 0.** Hardware lead time and key-pair provisioning are the long poles for the front-desk switch in Phase 4.
- Each desk needs a network drop or Wi-Fi for the terminal, the staff join page, and the desk's key pair stored server-side — never in the browser.
- Train staff the week before cutover; the old card field will be gone.

OWNERS AT A *glance*

WORKSTREAM	OWNER
Data model, UI, batch generator	Member-system vendor
Sweep execution and reconciliation	IntelliPay (Option A) · GHF IT + vendor (Option B)
Online join integration, staff capture tool	Us
Outreach to exception members	GHF member services
PCI attestation and purge	GHF IT / compliance

WORKSTREAM	OWNER
API keys, Account Updater, batch spec, sandbox ACH, terminal quote & provisioning	IntelliPay
Terminal purchase, network drops at desks, staff training	GHF operations

APPENDIX - PAYLOADS THAT COME BACK, *and what to store*

Field names are verbatim from IntelliPay's documentation and from responses observed against the sandbox merchant. Nothing that comes back contains a full card or bank number.

1 · Lightbox Store Only — browser callback `runOnApproval(r)` — the primary source of a token

```
r = {
  status,                // > 0 on success
  response: "A",         // A approved · D / E arrive via runOnNonApproval
  custid,                // ← THE TOKEN
  paymenttype,           // "C" card | "A" bank           (observed: "C")
  methodhint,            // "VI ***1111"           (observed)
  cardbrand,             // "Visa", "Mastercard", ...
  paymentid, authcode, declinereason, amount, fee, // only when a charge ran (not store-only)
  hmac, nonce,           // integrity check – verify, don't persist
  receiptelements        // receipt markup – ignore
}
```

2 · `cust_create` — the one-time sweep (Option B) or the bulk-import mapping file (Option A)

```
{ "status": 19682099 }           // > 0 → the custid (token) · -1 auth · -2 could not create
```

3 · `cust_read` — what IntelliPay holds against a token `OBSERVED`

```
{ "status": "19682099",          // = custid
  "account": "GHF-...",          // our member ID, as we sent it
  "firstname", "lastname", "address1", "address2", "city", "state", "zipcode", "country", "phone",
  "cardtype": "Card",            // brand text when a card is stored
  "cardending": "1111",          // last 4
  "cardnumdisplay": "...",       // masked display form
  "cardname", "expdate",         // MMY
  "bankacctnum": "???",          // masked
  "bankaccttype": "C"|"S", "routingnum",
  "relatedmerchantkey": "M7976" // the merchant the token belongs to
}
```

Use it to reconcile after the sweep and to refresh display hints — fetch on demand, don't mirror it.

4 · `card_payment` by `custid` — the online-join "due today" charge

```
success: { "status": 19866057, "response": "A", "authcode": "269224", "declinereason": "Success",
           "avsresult": "Y", "fee": 0.01, "custid": 18096987, "paymentid": 19866057 }
decline: { "status": -4, "response": "D", "authcode": "", "declinereason": "DECLINE", "paymentid": 19866057 }
```

Always read **both** `status` and `response`. A decline is still a recorded payment with a `paymentid`.

5 · `payment_create` / batch upload — the nightly dues run

Per row: `status` > 0 = `paymentid` queued. Failures: -2 `custid` not found · -3 bad type · -4 routing missing/invalid · -5 bank account missing · -6 bank type not C/S · -7 **stored card expired** · -8 stored card missing · -9 record not created · -10 amount invalid. ACH is asynchronous: the settle/return result arrives days later via postback.

6 · Postback — server-to-server, the durable record (form-encoded POST)

```
timestamp, customerid (= custid), paymentid, account, firstname, lastname,
address1, address2, city, state, zipcode, phone, method ("CARD"|"ACH"),
invoice, authcode, avsdata, ipaddress, amount, fee, total, comment, notes,
merchantid, origin, department, arglist
```

Up to 3 delivery attempts. The handler must be **idempotent** on `paymentid`, return 2xx only after the row is saved, and never redirect.

Table 1 · `member_billing` — one row per member

COLUMN	TYPE	COMES FROM	NOTES
<code>member_id</code>	string(20) PK	ours	Sent to IntelliPay as <code>account</code> on every call — the join key.
<code>ipay_custid</code>	string(32) TOKEN	Lightbox <code>custid</code> · <code>cust_create</code> <code>status</code> · postback <code>customerid</code> · bulk-import mapping	Required for autopay. Read-only in the UI.
<code>ipay_merchant_id</code>	string(12)	postback <code>merchantid</code> · <code>cust_read</code> <code>relatedmerchantkey</code>	Which MID the token lives under.
<code>token_type</code>	char(1) C/A	Lightbox <code>paymenttype</code> · postback <code>method</code> (CARD→C, ACH→A) · what we sent in the sweep	Batch file needs it.
<code>method_hint</code>	string(24)	Lightbox <code>methodhint</code> · <code>cust_read</code> <code>cardnumdisplay</code>	Display only.
<code>card_brand</code>	string(16)	Lightbox <code>cardbrand</code> · <code>cust_read</code> <code>cardtype</code>	Display; Account Updater covers Visa/MC only.

COLUMN	TYPE	COMES FROM	NOTES
card_last4	char(4)	cust_read · cardending	Display / reconciliation. Not cardholder data.
card_exp_mmyy	char(4)	cust_read · expdate	Allowed under PCI. Drives expiring-card outreach for Amex/Discover.
token_source	enum	ours	migration · online-join · front-desk · member-update
token_created_at	datetime	ours, at receipt	
token_status	enum	batch returns / postbacks	active · expired (-7) · needs-update (-2/-4/-5/-8) · declined
token_status_reason	string(64)	last declinereason or numeric code	For staff, not logic.
token_updated_at	datetime	ours	

Table 2 · payment_history — one row per charge attempt

COLUMN	TYPE	COMES FROM
ipay_paymentid	string(32) PK	response / postback paymentid — the idempotency key
member_id	string(20)	ours · postback account
ipay_custid	string(32)	response custid · postback customerid
invoice	string(40)	ours: GHF-<memberId>-<YYYYMMDD> — unique per member per cycle
amount, fee, total	decimal(10,2)	response / postback
method	CARD / ACH	postback method
response	char(1) A/D/E	response response
status_code	int	numeric status (paymentid or negative code)
authcode	string(12)	response / postback authcode
declinereason	string(64)	response declinereason
avsresult	char(1)	response avsresult · postback avsdata
origin	string(16)	postback origin — API / Batch Upload / Hosted Lightbox
source	enum	ours: lightbox · api · batch-return · postback
ipay_timestamp	datetime	postback timestamp (Mountain Time — convert)
received_at	datetime	ours
raw_payload	json	the full response/postback as received — audit trail; contains no card data

Never store: `cardnum`, `cvv`, `bankacctnum`, `routingnum`, `trackdata`, and the Lightbox `hmac / nonce` (verify, then discard). Nothing that comes back from IntelliPay contains a full number — the only way one lands in these tables is if someone types it in, which is why the old fields must be deleted, not just hidden.

Nightly SFTP row (what leaves GHF): `member_id` (as `account`) · `ipay_custid` · `token_type` · `amount` · `invoice` — column names to be confirmed against IntelliPay's batch spec (open question 2).